

6.7800 Project Part 2

Alan Duan

May 4, 2026

1 High-Level Overview

The decoder is structured as a two-step pipeline: (1) locate the breakpoint (if one is present), then (2) decode each segment independently using a MAP estimate of the substitution cipher key.

1.1 MAP Estimation via Iterated Greedy + MCMC

Given a ciphertext segment, we estimate the cipher key f (a permutation of the 28-character alphabet) by maximizing the log-likelihood

$$\log p(y \mid f) = \log p_0[f^{-1}(y_0)] + \sum_{k=1}^{n-1} \log T[f^{-1}(y_k), f^{-1}(y_{k-1})],$$

where p_0 is the English unigram distribution and T is the English bigram transition matrix ($T[\text{current}][\text{previous}]$).

Phase 1 — Frequency-based initialization. Rather than starting from the identity permutation, we sort ciphertext characters by empirical frequency and match them in rank order to English characters sorted by their prior probability. Since a substitution cipher preserves character frequencies, this places the chain very close to the true permutation from the outset.

Phase 2 — Multi-start greedy hill-climbing. From the frequency initialization we run a deterministic greedy search: iterate over all $\binom{28}{2} = 378$ pairwise swaps of the decoder and accept any swap that strictly increases the log-likelihood; repeat until no improving swap exists (a local optimum). We then perturb the best decoder found by randomly applying 2–4 random transpositions and re-run the greedy climb. This is repeated 100 times,

keeping the best result across all restarts. Each restart is cheap because log-likelihood evaluation uses precomputed bigram counts (see below), so all 100 restarts complete in well under one second.

Phase 3 — Metropolis-Hastings MCMC warm start. Starting from the best greedy solution we run standard Metropolis-Hastings with random pairwise-swap proposals for 100,000 iterations (20,000 burn-in), recording the highest-likelihood permutation seen post-burn-in. Because the greedy phase already places the chain near the global optimum, MH serves as a fine-grained explorer that can escape rare remaining local optima.

1.2 Fast Log-Likelihood via Bigram Counts

The key engineering improvement over Part I is replacing the $O(n)$ Python loop in every log-likelihood evaluation with a single $O(|\Sigma|^2)$ vectorised numpy operation. We precompute once the bigram count matrix $B \in \mathbb{Z}^{|\Sigma| \times |\Sigma|}$ where $B[i][j]$ is the number of times cipher character i is immediately followed by j . Then

$$\sum_{k=1}^{n-1} \log T[f^{-1}(y_k), f^{-1}(y_{k-1})] = \sum_{i,j} B[i][j] \cdot \log T[f^{-1}(j), f^{-1}(i)],$$

which reduces to a permuted matrix lookup and element-wise sum—roughly $100\times$ faster than iterating over n characters in Python for $n \approx 2000$, $|\Sigma| = 28$. This speedup funds 100 greedy restarts and 100,000 MH steps within a 5–11 second wall-clock budget per test case.

2 Enhancements to the Part I Solution

Part I used plain Metropolis-Hastings initialized from the identity permutation with 10,000 burn-in and 10,000 post-burn-in iterations. The enhancements are:

- **Frequency initialization** (Phase 1) replaces the identity start, placing the chain near the solution immediately.
- **Multi-start greedy** (Phase 2) efficiently finds and escapes local optima without relying on random walk mixing.

- $O(|\Sigma|^2)$ **bigram LL** reduces per-evaluation cost by $\sim 100\times$, enabling 100 restarts and 100,000 MH steps within the time previously used for 10,000 steps from a cold start.

3 Handling Breakpoints

3.1 Breakpoint Localization via Bigram Mutual Information

The mutual information between adjacent characters,

$$\text{MI}(X_k, X_{k+1}) = \sum_{i,j} p(i,j) \log \frac{p(i,j)}{p(i)p(j)},$$

is *permutation-invariant*: for any single substitution cipher applied to English text it equals the English bigram MI, regardless of which permutation was used. A segment that spans the true breakpoint contains two inconsistent transition structures and scores lower MI than either pure half.

We scan all $n + 1$ candidate split positions in one $O(n \cdot |\Sigma|^2)$ pass using prefix bigram count matrices. The score for split s is the weighted sum

$$\text{score}(s) = N_L \cdot \text{MI}(B_L) + N_R \cdot \text{MI}(B_R),$$

where $N_L = s - 1$ and $N_R = n - s - 1$ are the bigram counts in each half. This is the log-likelihood ratio statistic for a Markov chain changepoint test. Positions with fewer than $\max(50, n/10)$ characters on either side are excluded to avoid noise from sparse bigram estimates. The argmax gives a candidate breakpoint C with no MCMC required.

3.2 Fixed-Cipher Refinement

We run full MCMC on $y_{<C}$ and $y_{\geq C}$ to obtain approximate cipher keys $\hat{f}_L, \hat{f}_R$. Because C is close to the true breakpoint B , the contamination (characters from the wrong cipher in each half) is small ($\approx |C - B|/n$), so the estimated keys are approximately correct.

We then evaluate

$$\log p(y_{<s} \mid \hat{f}_L) + \log p(y_{\geq s} \mid \hat{f}_R)$$

for every s in the window $[C-r, C+r]$ with radius $r = \max(100, 3\sqrt{n})$, using the *fixed* estimated ciphers—no additional MCMC. This $O(r \cdot n)$ sweep is fast and accurately localizes the breakpoint because the log-likelihood sharply penalizes even a single character being decoded with the wrong cipher. The final plaintext is obtained by decoding $y_{<s^*}$ under $\hat{f}_L$ and $y_{\geq s^*}$ under $\hat{f}_R$.

4 R&D Process

Testing. We used the provided `test.py` script together with the sample plaintext and ciphertext files, tracking character match rate (correct characters / total characters) as the primary metric. We also generated additional test cases by running `encode.py` on other texts with random seeds to check robustness.

Iteration history. An early approach iterated over all n possible breakpoint positions, running MCMC on each half—far too slow. Binary search was attempted next but failed because the joint log-likelihood is not unimodal over split positions. Cheap MCMC (100–500 iterations) was tried as a fast scoring proxy, but it consistently failed to converge on segments of length ~ 1000 characters, producing near-random scores and thus random split selection (14/2017 characters correct on the breakpoint test).

The key insight that fixed this was that MCMC is unnecessary for split *scoring*: the MI statistic identifies the breakpoint region without any MCMC, and fixed-cipher LL evaluation precisely locates it within the window without additional MCMC. MCMC is needed only *once* per segment, for the final cipher estimation.

Performance focus. The $O(|\Sigma|^2)$ bigram LL was the single most impactful optimization: it reduced wall-clock time per test case from ~ 21 s to ~ 5 s while enabling a $10\times$ increase in both greedy restarts and MH iterations. The frequency initialization and greedy multi-start together eliminated most remaining decoding errors by finding a much better warm start than the identity permutation.